

JavaScript Problem Solving - Extended (For SSE Interviews)

Problem 1: Two Sum (Find Pair with Target Sum)

Problem Statement: Given an array of integers and a target value, return the indices of the two numbers such that they add up to the target.

Solution & Explanation: Use a hash map to store the complement of each number. For each element, check if its complement exists in the map.

Code:

```
javascript
function twoSum(nums, target) {
    const map = new Map();
    for (let i = 0; i < nums.length; i++) {
        let complement = target - nums[i];
        if (map.has(complement)) return [map.get(complement), i];
        map.set(nums[i], i);
    }
}
```

Problem 2: Remove Duplicates Without Using Set

Problem Statement: Remove duplicates from an array while preserving the original order of elements.

Solution & Explanation: Use an array and includes() method to build the result by checking if each element already exists in the result array.

Code:

```
javascript
function removeDuplicates(arr) {
    let result = [];
    for (let i = 0; i < arr.length; i++) {
        if (!result.includes(arr[i])) result.push(arr[i]);
    }
    return result;
}
```

```
}
```

Problem 3: Missing Number Using Gauss' Formula

Problem Statement: Find a missing number in an array that should contain numbers from 1 to n.

Solution & Explanation: Use Gauss' formula to calculate the expected sum of numbers from 1 to n, then subtract the actual sum to find the missing number.

Code:

```
javascript
function findMissing(arr, n) {
    const expected = (n * (n + 1)) / 2;
    const actual = arr.reduce((a, b) => a + b, 0);
    return expected - actual;
}
```

Problem 4: Move Zeroes to End Without Built-ins

Problem Statement: Move all zeroes in an array to the end while maintaining the relative order of non-zero elements, in-place.

Solution & Explanation: Use two pointers - one to track position for non-zero elements, another to iterate through the array.

Code:

```
javascript
function moveZeroes(arr) {
    let index = 0;
    for (let i = 0; i < arr.length; i++) {
        if (arr[i] !== 0) arr[index++] = arr[i];
    }
    while (index < arr.length) arr[index++] = 0;
    return arr;
}
```

Problem 5: Bit Manipulation: Find the Unique Number

Problem Statement: In an array where every element appears twice except one, find the element that appears only once.

Solution & Explanation: Use XOR operation. Since XOR of a number with itself is 0, all paired numbers will cancel out, leaving only the unique number.

Code:

```
javascript
function findUnique(arr) {
    let result = 0;
    for (let i = 0; i < arr.length; i++) result ^= arr[i];
    return result;
}
```

Problem 6: Reverse a String Without Using reverse()

Problem Statement: Reverse a string without using the built-in reverse() method.

Solution & Explanation: Iterate through the string from end to beginning and build the reversed string character by character.

Code:

```
javascript
function reverseString(str) {
    let reversed = "";
    for (let i = str.length - 1; i >= 0; i--) reversed += str[i];
    return reversed;
}
```

Problem 7: Check for Palindrome

Problem Statement: Check if a given string is a palindrome (reads the same forwards and backwards).

Solution & Explanation: Use two pointers approach - one from start and one from end, comparing characters while moving towards center.

Code:

```
javascript
function isPalindrome(str) {
```

```

    let start = 0, end = str.length - 1;
    while (start < end) {
        if (str[start] !== str[end]) return false;
        start++;
        end--;
    }
    return true;
}

```

Problem 8: Factorial Using Recursion

Problem Statement: Calculate the factorial of a number using recursion.

Solution & Explanation: Use recursive approach where $\text{factorial}(n) = n * \text{factorial}(n-1)$, with base case when n is 0 or 1.

Code:

```

javascript
function factorial(n) {
    if (n === 0 || n === 1) return 1;
    return n * factorial(n - 1);
}

```

Problem 9: Fibonacci Series

Problem Statement: Generate the first n numbers of the Fibonacci sequence.

Solution & Explanation: Use iterative approach to build the sequence where each number is the sum of the two preceding ones.

Code:

```

javascript
function fibonacci(n) {
    let arr = [0, 1];
    for (let i = 2; i < n; i++) arr[i] = arr[i - 1] + arr[i - 2];
    return arr.slice(0, n);
}

```

Problem 10: Find the Largest Number in an Array

Problem Statement: Find the maximum value in an array without using built-in Math.max() function.

Solution & Explanation: Iterate through the array comparing each element with the current maximum value.

Code:

```
javascript
function findMax(arr) {
    let max = arr[0];
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
```

Problem 11: Count Vowels in a String

Problem Statement: Count the number of vowels (both uppercase and lowercase) in a given string.

Solution & Explanation: Iterate through each character and check if it exists in the vowels string.

Code:

```
javascript
function countVowels(str) {
    let count = 0;
    let vowels = "aeiouAEIOU";
    for (let char of str) {
        if (vowels.includes(char)) count++;
    }
    return count;
}
```

Problem 12: Capitalize First Letter of Each Word

Problem Statement: Capitalize the first letter of each word in a sentence while making the rest lowercase.

Solution & Explanation: Split the string into words, transform each word, then join them back together.

Code:

```
javascript
function capitalizeWords(str) {
    let words = str.split(" ");
    let result = words.map(word => word[0].toUpperCase() +
word.slice(1).toLowerCase());
    return result.join(" ");
}
```

Problem 13: Check if a Number is Prime

Problem Statement: Determine if a given number is prime (only divisible by 1 and itself).

Solution & Explanation: Check divisibility from 2 to square root of the number for optimization.

Code:

```
javascript
function isPrime(n) {
    if (n <= 1) return false;
    for (let i = 2; i * i <= n; i++) {
        if (n % i === 0) return false;
    }
    return true;
}
```

Problem 14: Find Second Largest Number in Array

Problem Statement: Find the second largest number in an array.

Solution & Explanation: Keep track of the largest and second largest numbers in a single pass through the array.

Code:

```
javascript
function secondLargest(arr) {
    let max = -Infinity, second = -Infinity;
    for (let num of arr) {
```

```

        if (num > max) {
            second = max;
            max = num;
        } else if (num > second && num !== max) {
            second = num;
        }
    }
    return second;
}

```

Problem 15: Count Occurrences of Each Character

Problem Statement: Count how many times each character appears in a string.

Solution & Explanation: Use an object to store character counts, incrementing the count for each character encountered.

Code:

```

javascript
function countCharacters(str) {
    let counts = {};
    for (let char of str) counts[char] = (counts[char] || 0) + 1;
    return counts;
}

```

Problem 16: Flatten a Nested Array

Problem Statement: Flatten a multi-dimensional array into a single-dimensional array.

Solution & Explanation: Use recursion to handle nested arrays at any depth level.

Code:

```

javascript
function flatten(arr) {
    let result = [];
    for (let item of arr) {
        if (Array.isArray(item)) result =
result.concat(flatten(item));
        else result.push(item);
    }
    return result;
}

```

```
}
```

Problem 17: Anagram Check

Problem Statement: Check if two strings are anagrams (contain the same characters in different order).

Solution & Explanation: Sort both strings and compare them - anagrams will have identical sorted strings.

Code:

```
javascript
function isAnagram(str1, str2) {
    return str1.split('').sort().join('') ===
str2.split('').sort().join('');
}
```

Problem 18: Find GCD (Greatest Common Divisor)

Problem Statement: Find the greatest common divisor of two numbers using Euclidean algorithm.

Solution & Explanation: Use recursive approach where $GCD(a,b) = GCD(b, a \% b)$ until one number becomes 0.

Code:

```
javascript
function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
}
```

Problem 19: Find Longest Word in a Sentence

Problem Statement: Find the longest word in a given sentence.

Solution & Explanation: Split the sentence into words and use reduce to find the word with maximum length.

Code:

javascript

```
function longestWord(sentence) {
    let words = sentence.split(' ');
    return words.reduce((longest, word) => word.length >
longest.length ? word : longest, "");
}
```

Problem 20: Find Intersection of Two Arrays

Problem Statement: Find common elements between two arrays.

Solution & Explanation: Use a Set for the first array for $O(1)$ lookup, then filter the second array for elements present in the Set.

Code:

javascript

```
function intersection(arr1, arr2) {
    const set = new Set(arr1);
    return arr2.filter(item => set.has(item));
}
```

Problem 21: Check for Balanced Parentheses

Problem Statement: Check if parentheses, brackets, and braces are properly balanced in a string.

Solution & Explanation: Use a stack to track opening brackets and match them with corresponding closing brackets.

Code:

javascript

```
function isBalanced(str) {
    const stack = [];
    const map = { '(': ')', '[': ']', '{': '}' };
    for (let char of str) {
        if (map[char]) stack.push(map[char]);
        else if (Object.values(map).includes(char)) {
            if (stack.pop() !== char) return false;
        }
    }
    return stack.length === 0;
}
```

```
}
```

Problem 22: Convert Roman Numerals to Integer

Problem Statement: Convert a Roman numeral string to its integer equivalent.

Solution & Explanation: Use a mapping of Roman numerals to integers. If current value is less than next value, subtract it; otherwise, add it.

Code:

```
javascript
function romanToInt(s) {
    const map = { I:1, V:5, X:10, L:50, C:100, D:500, M:1000 };
    let total = 0;
    for (let i = 0; i < s.length; i++) {
        const curr = map[s[i]], next = map[s[i+1]];
        total += (curr < next) ? -curr : curr;
    }
    return total;
}
```

Problem 23: Find All Pairs That Sum to Target

Problem Statement: Find all unique pairs of numbers in an array that sum to a target value.

Solution & Explanation: Use a Set to track seen numbers and another Set to store unique pairs to avoid duplicates.

Code:

```
javascript
function pairSum(arr, target) {
    const seen = new Set(), pairs = new Set();
    for (let num of arr) {
        let complement = target - num;
        if (seen.has(complement)) {
            pairs.add([Math.min(num, complement), Math.max(num, complement)].toString());
        }
        seen.add(num);
    }
    return Array.from(pairs).map(p => p.split(',').map(Number));
}
```

```
}
```

Problem 24: Remove Falsy Values from an Array

Problem Statement: Remove all falsy values (false, null, 0, "", undefined, NaN) from an array.

Solution & Explanation: Iterate through the array and only add truthy values to the result array.

Code:

```
javascript
function removeFalsy(arr) {
    const result = [];
    for (let val of arr) {
        if (val) result.push(val);
    }
    return result;
}
```

Problem 25: Rotate an Array by k Steps

Problem Statement: Rotate an array to the right by k steps.

Solution & Explanation: Use array reversal technique: reverse entire array, then reverse first k elements, then reverse remaining elements.

Code:

```
javascript
function rotateArray(arr, k) {
    k = k % arr.length;
    function reverse(start, end) {
        while (start < end) [arr[start], arr[end]] = [arr[end],
arr[start]], start++, end--;
    }
    reverse(0, arr.length - 1);
    reverse(0, k - 1);
    reverse(k, arr.length - 1);
    return arr;
}
```

Problem 26: Equilibrium Index in an Array

Problem Statement: Find an index where the sum of elements on the left equals the sum of elements on the right.

Solution & Explanation: Calculate total sum first, then iterate while maintaining left sum and checking if it equals right sum.

Code:

```
javascript
function findEquilibriumIndex(arr) {
    const totalSum = arr.reduce((acc, val) => acc + val, 0);
    let leftSum = 0;
    for (let i = 0; i < arr.length; i++) {
        if (leftSum === totalSum - leftSum - arr[i]) return i;
        leftSum += arr[i];
    }
    return -1;
}
```

Problem 27: Birthday Cake Candles (Find Max Frequency)

Problem Statement: Count how many candles have the maximum height (can be blown out).

Solution & Explanation: Find the maximum height, then count how many candles have that height.

Code:

```
javascript
function birthdayCakeCandles(candles) {
    const max = Math.max(...candles);
    return candles.filter(h => h === max).length;
}
```

Problem 28: R-Type Array Game Pattern

Problem Statement: Determine if you can exit an array-based game where each element represents the number of steps to move forward.

Solution & Explanation: Simulate the game by following the steps. Mark visited positions as 0. Exit if you go out of bounds or get stuck in a zero loop.

Code:

javascript

```
function willExit(arr) {  
  let i = 0;  
  while (i >= 0 && i < arr.length) {  
    let step = arr[i];  
    arr[i] = 0;  
    i += step;  
    if (step === 0) return false;  
  }  
  return i < 0 || i >= arr.length;  
}
```